

Install + Core Imports

```
import { initializeApp }
  from "firebase/app";
import { getAuth,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signInOut, onAuthStateChanged, User }
  from "firebase/auth";
import { getFirestore, collection,
  doc, addDoc, setDoc, getDoc,
  getDocs, onSnapshot, updateDoc,
  deleteDoc, DocumentData }
  from "firebase/firestore";
```

firebaseConfig.ts

```
const firebaseConfig = {
  apiKey: "...",
  authDomain: "...",
  projectId: "...",
  storageBucket: "...",
  messagingSenderId: "...",
  appId: "...",
};

const app = initializeApp(firebaseConfig);
const firebaseAuth = getAuth(app);
const fireDB = getFirestore(app);

export { app, firebaseAuth, fireDB };
```

Firestore Type

```
export type Book = {
  id: string;    // Firestore doc ID
  title: string;
  author: string;
  genre: string;
  uid: string;   // owner Auth UID
};
```

Auth Listener Custom Hook

```
import { onAuthStateChanged, User }
  from "firebase/auth";
import { useEffect, useState }
  from "react";
import { firebaseAuth }
  from "../firebaseConfig";

export function userAuthentication() {
  const [user, setUser] =
    useState<User | null | undefined>(
      undefined
    );

  useEffect(() => {
    const unsubscribe =
      onAuthStateChanged(
        firebaseAuth,
        (currentUser) => {
          setUser(currentUser);
        }
      );

    return unsubscribe;
  }, []);

  return user;
}
```

Auth Route Guard

```
import { userAuthentication }
  from "@userAuthentication";
import { Redirect } from "expo-router";

export default function Index() {
```

```
  const user = userAuthentication();

  if (user === undefined) {
    return null;    // still loading
  }

  if (user === null) {
    return <Redirect href="/signIn" />;
  }

  return <Redirect href="/BookList" />;
}
```

Router Layouts

```
// app/_layout.tsx
import { Stack } from "expo-router";
export default function RootLayout() {
  return <Stack screenOptions={{
    headerShown: false
  }} />;
}

// app/(auth)/_layout.tsx
export default function AuthLayout() {
  return (
    <Stack>
      <Stack.Screen name="signIn"
        options={{ title: "Sign In" }} />
      <Stack.Screen name="signUp"
        options={{ title: "Sign Up" }} />
    </Stack>
  );
}
```

Object Form State

```
const [userObject, setUserObject] =
  useState({
    email: "",
    password: "",
    confirmPassword: "",
    fullname: "",
    contact: "",
    location: "",
    error: "",
  });

// update ONE field; preserve others
setUserObject({
  ...userObject,
  email: text,
});
```

Sign Up + Save Profile

```
const router = useRouter();

const onSignUp = async () => {
  if (userObject.email === "" ||
    userObject.password === "") {
    setUserObject({
      ...userObject,
      error: "Email/password required",
    });
    return;
  }

  try {
    const credential = await
      createUserWithEmailAndPassword(
        firebaseAuth,
        userObject.email,
        userObject.password
      );

    const profile = {
```

```
    name: userObject.fullname,
    email: userObject.email,
    contact: userObject.contact,
    location: userObject.location,
  };

  await setDoc(
    doc(fireDB, "userProfile",
      credential.user.uid),
    profile
  );

  router.replace("/BookList");
} catch (error: any) {
  setUserObject({
    ...userObject,
    error: error.message,
  });
}
};
```

Sign-Up Inputs

```
<TextInput
  value={userObject.email}
  onChangeText={(text) =>
    setUserObject({
      ...userObject, email: text
    })
  }
  keyboardType="email-address"
  autoCapitalize="none"
/>

{!userObject.error && (
  <Text>{userObject.error}</Text>
)}

<TouchableOpacity onPress={onSignUp}>
  <Text>Sign Up</Text>
</TouchableOpacity>
```

Sign In

```
const onSignIn = async () => {
  if (!userObject.email ||
    !userObject.password) {
    setUserObject({
      ...userObject,
      error: "Email/password required",
    });
    return;
  }

  try {
    const result = await
      signInWithEmailAndPassword(
        firebaseAuth,
        userObject.email,
        userObject.password
      );

    Alert.alert(
      "Success",
      `Welcome ${result.user.email}`,
      [{ text: "Okay", onPress: () =>
        router.replace("/BookList") }]
    );
  } catch (error: any) {
    setUserObject({
      ...userObject,
      error: error.message,
    });
  }
};
```

Add Firestore Document

```
const [book, setBook] = useState<Book>({
  id: "", title: "", author: "",
  genre: "", uid: "",
});

const addBook = async () => {
  if (!book.title || !book.author ||
    !book.genre) {
    Alert.alert("Error", "Fill all fields");
    return;
  }

  try {
    const newBook = {
      title: book.title,
      author: book.author,
      genre: book.genre,
      uid: firebaseAuth.currentUser?.uid,
    };

    const collectionRef =
      collection(fireDB, "BookDB");
    const docRef = await addDoc(
      collectionRef, newBook
    );

    Alert.alert("Added", docRef.id);
    setBook({
      ...book, title: "", author: "",
      genre: ""
    });
  } catch (error) {
    console.log(error);
  }
};
```

One-Time Profile Read

```
const [profile, setProfile] =
  useState<DocumentData>({
    email: "N/A", name: "N/A",
    contact: "N/A", location: "N/A",
  });

const getUserProfile = async () => {
  const uid = firebaseAuth.currentUser!.uid;
  const profileRef =
    doc(fireDB, "userProfile", uid);
  const docSnap = await getDoc(profileRef);

  if (docSnap.exists()) {
    setProfile(docSnap.data());
  } else {
    Alert.alert("No profile found");
  }
};

useEffect(() => {
  getUserProfile();
}, []);
```

Real-Time Collection Read

```
const [bookList, setBookList] =
  useState<Book[]>([]);

useEffect(() => {
  const unsubscribe = onSnapshot(
    collection(fireDB, "BookDB"),
    (snapshot) => {
      const books: Book[] =
        snapshot.docs.map((bookDoc) => ({
          id: bookDoc.id,
          title: bookDoc.data().title,
          author: bookDoc.data().author,
          genre: bookDoc.data().genre,
          uid: bookDoc.data().uid,
        }));

      setBookList(books);
    }
  );

  return unsubscribe;
}, []);
```

FlatList + Item Component

```
<FlatList
  data={bookList}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <View>
      <Text>{item.title}</Text>
      <Text>{item.author}</Text>
    </View>
  )}
```

```
  )}
/>

Update Document

const bookRef =
  doc(fireDB, "BookDB", book.id);

const updateBook = async () => {
  try {
    await updateDoc(bookRef, {
      title: 'Updated ${book.title}',
      author: 'Updated ${book.author}',
      genre: 'Updated ${book.genre}',
    });
  } catch (error) {
    console.log(error);
  }
};
```

updateDoc updates ONLY listed fields;
other document fields stay unchanged.

Delete Document + Alert

```
const deleteBook = () => {
  Alert.alert(
    book.title,
    "Delete this book?",
    [
      { text: "No", style: "cancel" },
      {
        text: "Yes",
        onPress: async () => {
          await deleteDoc(bookRef);
        },
      },
    ],
  );
};
```

// optional owner-only UI/action guard
const isOwner =
 firebaseAuth.currentUser?.uid ===
 book.uid;

```
{isOwner && (
  <TouchableOpacity onPress={deleteBook}>
    <Text>Delete</Text>
  </TouchableOpacity>
)}
```

Tabs + Sign Out Header

```
import { Tabs, useRouter }
  from "expo-router";
import MaterialIcons
  from "@expo/vector-icons/MaterialIcons";
import { signOut } from "firebase/auth";
import { firebaseAuth }
  from "@firebase/config";
```

```
export default function TabsLayout() {
  const router = useRouter();
```

```
  return (
    <Tabs>
      <Tabs.Screen name="BookList"
        options={{
          title: "Books",
          tabBarActiveTintColor: "orange",
          tabBarIcon: ({ color }) =>
            <MaterialIcons name="home"
              size={24} color={color} />,
          headerRight: () =>
            <MaterialIcons
              name="exit-to-app" size={24}
              onPress={async () => {
                await signOut(firebaseAuth);
                router.replace("/signIn");
              }}
            />,
        }}
      />

      <Tabs.Screen name="NewBook"
        options={{ title: "Add Book" }} />
      <Tabs.Screen name="UserProfile"
        options={{ title: "Profile" }} />
    </Tabs>
  );
}
```

Camera Imports + State

```
import { CameraType, CameraView,
  useCameraPermissions }
  from "expo-camera";
import * as MediaLibrary
```

```
  from "expo-media-library";
import { useEffect, useRef, useState }
  from "react";
import { Alert, Button, Text }
  from "react-native";

const [cameraPermission,
  requestCameraPermission] =
  useCameraPermissions();

const [mediaPermission,
  requestMediaPermission] =
  MediaLibrary.usePermissions({
    granularPermissions: ["photo"],
  });

const cameraRef = useRef<CameraView>(null);
const isCameraReady = useRef(false);
const [cameraType, setCameraType] =
  useState<CameraType>("back");
```

Request Both Permissions

```
const checkPermission = async () => {
  await requestCameraPermission();

  try {
    await requestMediaPermission();
  } catch (error) {
    console.log("Media request skipped");
  }
};

useEffect(() => {
  checkPermission();
}, []);
```

camera permission -> use camera
media permission -> save to gallery
They are SEPARATE permissions.

Permission UI

```
{cameraPermission?.granted && (
  <CameraView
    ref={cameraRef}
    style={{ flex: 1 }}
    facing={cameraType}
    onCameraReady={() => {
      isCameraReady.current = true;
    }}
  />
)}
```

Only render CameraView after permission.

Flip Camera

```
const toggleCameraType = () => {
  setCameraType((current) =>
    current === "back" ? "front" : "back"
  );
};

<Button title="Flip Camera"
  onPress={toggleCameraType} />
```

Take + Save Photo

```
const takePhoto = async () => {
  try {
    if (!mediaPermission?.granted) {
      Alert.alert("Gallery permission needed");
      return;
    }

    if (!cameraRef.current ||
      !isCameraReady.current) {
      console.log("Camera not ready");
      return;
    }

    const photo = await
      cameraRef.current.takePictureAsync();

    await MediaLibrary.createAssetAsync(
      photo.uri
    );

    Alert.alert("Saved to gallery");
  } catch (error) {
    console.log(error);
  }
};
```

```
<Button title="Take Photo"
  onPress={takePhoto} />
```